

ДИСЦИПЛИНА	Программирование (полное наименование дисциплины без сокращений)
ИНСТИТУТ	Институт тонких химических технологий им. М.В. Ломоносова
КАФЕДРА	Кафедра информационных систем в химической технологии полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические занятия (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Серебренников Н.П. (фамилия, имя, отчество)
СЕМЕСТР	2-й семестр, 1-й курс бакалавриата, 2024-2025 учебный год (указать семестр обучения, учебный год)

Практическая работа № 8. Отладка кода, профилирование, использование юнит-тестов.

Цель работы

Отработать механизмы отладки кода в Python, научиться использовать юнит-тесты и профилирование для создания более эффективного и отказоустойчивого кода.

Задания

1. Отладка кода с использованием инструментов Spyder IDE

- Найдите и исправьте логическую ошибку в программе, которая должна суммировать все элементы списка до (и включая) второго отрицательного числа. Ошибка проявляется только в определённых случаях. Код программы в файле **test_debug_wrong.py**.
- Найдите и исправьте логическую ошибку в файле **test_debug_wrong2.py**.

Расчет итоговой оценки студента по заданиям и экзамену

- Условие:

- Есть список оценок за задания и отдельно балл за экзамен. Итоговая оценка считается по следующей логике:
- Средний балл за задания учитывается с весом 0.4 (40%)
- Балл за экзамен — с весом 0.6 (60%)
- Если студент пропустил более одного задания (оценка 0), итоговая оценка снижается на 10%
- Найдите и исправьте ошибку в файле **test_debug_wrong3.py**. Программа считает слова, в которых есть хотя бы одна гласная.

2. Профилирование кода с помощью cProfile

- Дана программа, вычисляющая количество простых чисел до заданного числа n . Измерьте её производительность с помощью cProfile, а затем ускорьте, заменив неэффективный алгоритм.

```
def is_prime(n):  
    if n < 2:  
        return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
  
def count_primes(limit):  
    count = 0  
    for i in range(2, limit):  
        if is_prime(i):  
            count += 1  
    return count  
  
print(count_primes(10000))
```

- Запустите профилирование с помощью cProfile:

```
import cProfile
cProfile.run('count_primes(10000)')
```

- Найдите «узкие места» (например, is_prime вызывается слишком часто и работает медленно).
- Оптимизируйте алгоритм is_prime, например, проверяя деление только до $\sqrt{n}$
- Повторно измерьте производительность и сравните результаты.
- Посмотрите на сколько уменьшилось время выполнения.

3. Юнит-тестирование с использованием pytest

Установите pytest с помощью команды (вводить в командной строке)

```
pip install pytest
```

- Создайте простой модуль **math_utils.py** с функцией:

```
def add(a, b):
    return a + b
```

- Создайте файл тестов test_math_utils.py:

```
from math_utils import add

def test_add_positive():
    assert add(2, 3) == 5

def test_add_zero():
    assert add(0, 5) == 5

def test_add_negative():
    assert add(-1, -1) == -2
```

- Запустите тесты (выполнить команду необходимо из той же папки, где находятся файлы):

```
pytest
```

- Измените `add()` на неверную реализацию (например, `a - b`) и убедитесь, что тесты "падают".

3.1 Напишите утилиту для валидации паролей и юнит-тест для неё.

Создайте модуль `password_utils.py`, содержащий функции для проверки корректности паролей по следующим правилам:

Пароль должен:

- быть длиной от 8 до 20 символов;
- содержать хотя бы одну строчную букву;
- содержать хотя бы одну заглавную букву;
- содержать хотя бы одну цифру;
- не содержать пробелов;
- не состоять из одного повторяющегося символа (например, `aaaaaaaa` — нельзя);
- не быть в списке запрещённых паролей (например: `"password"`, `"123456"`, `"qwerty"` — передаются отдельным аргументом).

В `password_utils.py` реализовать функцию:

```
def is_valid_password(password: str, banned_list:
list[str]) -> bool:
    if not (8 <= len(password) <= 20):
        return False
    ...
```

Используйте функции: `isspace()` – проверка строки на пробелы, `isdigit()` – проверка символа на принадлежность к цифрам, `lower()` – перевод строки в нижний регистр, `upper()` – перевод строки в верхний регистр.

Используйте конструкции вида `if any( )` – если любое из условий в скобках `True`, `if all()` – если все из условий в скобках `True`

Напишите программу юнит-тестирования.

В `test_password_utils.py`:

- Напишите **не менее 15 тестов**, включая:
 - валидные пароли;
 - пароли слишком короткие/длинные;
 - без заглавной или без строчной буквы;
 - без цифр;
 - с пробелами;
 - повторяющиеся символы;
 - входящие в `banned_list`;

- проверку регистра в `banned_list` (если нужно).

```
from password_utils import is_valid_password

banned = ["password", "123456", "qwerty"]

def test_valid_password():
    assert is_valid_password("GoodPass1", banned)

def test_too_short():
    assert not is_valid_password("___", banned)
```

и т.д.

3.2 Реализуйте модуль `grades_utils.py`, содержащий следующие функции:

```
get_average(grades: list[float]) -> float
```

Возвращает среднее значение из списка оценок, округлённое до 2 знаков после запятой. Если список пуст, возвращает 0.0.

```
get_letter_grade(average: float) -> str
```

Преобразует среднюю оценку в буквенную:

- 90+ → 'A'
- 80–89 → 'B'
- 70–79 → 'C'
- 60–69 → 'D'
- <60 → 'F'

```
filter_failed_students(student_grades: dict[str, list[float]]) -> list[str]
```

Получает словарь вида:

```
{
    "Alice": [95, 88],
    "Bob": [59, 60],
    "Charlie": [40, 55]
}
```

Возвращает список имён студентов, чей средний балл < 60 .

```
get_top_student(student_grades: dict[str,  
list[float]]) -> str
```

Возвращает имя студента с наивысшим средним баллом. Если словарь пуст, вернуть строку "No students".

Требуется:

Создать отдельный файл **test_grades_utils.py** и:

- Написать по несколько тестов для каждой функции, включая граничные случаи;
- Проверить работу при пустом списке/словаре;
- Проверить, что функции правильно округляют, сравнивают и возвращают ожидаемые значения.